Git & GitHub Quick Guide

For working as a pair on the Kalman-filter track-fitting project

DAILY ROUTINE: every time you sit down to code

Follow these steps in order. If you only remember one page of this guide, make it this one.

1	**Open the project** Go to the repo folder and activate the environment.	`cd C:\Users\you\Projects\kalman-fit` `conda activate env_py312_kalman`
2	**Update main** Get everything your teammates have merged since yesterday.	`git checkout main` `git pull`

3	**Go to your branch**
	New task: create a branch.
	Continuing: switch to it and bring in the latest main.
		`git checkout -b feature/new-task  # new task`
		`git checkout feature/my-task  # continuing`
		`git merge main`

4	**Code and commit often**
	Small commits with clear messages. Check what you are saving first.
		`git status`
		`git add .`
		`git commit -m "Add process noise matrix"`

5	**Push before you stop**
	So your work is safe on GitHub and the others can see it.
		`git push`
		`# first time on a new branch:`
		`git push -u origin feature/my-task`

6	**Task finished?** Open a **Pull Request** on GitHub, a teammate reviews it, then **Merge**.	Then back to step 2 for the next task.

Never work directly on main. If something goes wrong, do not use `--force`: ask for help or use `git revert` (section 7).

1. The basic ideas

Git stores your project's history as a series of “snapshots” (commits). GitHub is the website that keeps a copy of that history so you can share it. These are the concepts you need:

Concept	What it is, in a few words
Repository (repo)	The project folder plus its whole history (stored in the hidden folder `.git`).
Commit	A snapshot of the project at a given moment, with a message explaining what changed.
Branch	A parallel line of work. It is a **complete** copy of the project where you make changes without touching `main`.
main	The main branch. It should always contain a version that works.
origin	The name Git gives to the copy of the repo that lives on GitHub.
push / pull	*push*: upload your commits to GitHub. *pull*: download the new commits from GitHub.

Concept	What it is, in a few words
Merge	Combine the changes from one branch into another.
Pull Request (PR)	A request on GitHub to merge your branch into `main`, so the other person can review it first.

2. Initial setup (only once)

Tell Git who you are. Use the same email as your GitHub account:

```
git config --global user.name "Alvaro Martinez"
git config --global user.email "you@email.com"
```

Clone the repo into a folder **outside OneDrive** and go into it:

```
cd C:\Users\alvar\Projects
git clone https://github.com/your-username/repo-name.git
cd repo-name
```

Create the project environment (Miniforge works the same as Anaconda):

```
conda env create -f environment.yml
conda activate env_py312_kalman
```

Careful. OneDrive syncs the `.git` folder while Git is modifying it, which can lock or duplicate files. GitHub already acts as a backup, so you don't need OneDrive.

- **Password:** GitHub does not accept your normal password when you push. Use a personal access token (Settings → Developer settings → Personal access tokens) or set up an SSH key.
- **Terminal:** on Windows, Git Bash accepts Linux commands (`ls`, `ll`, `cat`). In PowerShell use `ls -Force` to see hidden files. The `git` commands work the same everywhere.
- **Adding your teammate:** in the repo, Settings → Collaborators → Add people. They have to accept the invitation by email.

3. The basic cycle: change, save, upload

This is the cycle you will repeat constantly:

```
git status                 # what has changed?
git add file.py            # stage one file for the commit
git add .                  # ... or stage all changes
git commit -m "Add Kalman filter prediction step"
git push                   # upload to GitHub
```

And to download whatever your teammate has uploaded:

```
git pull
```

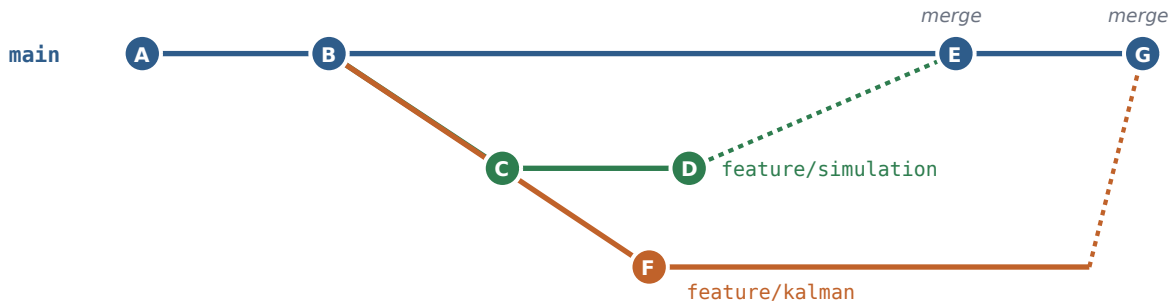
How to write good commits

- **Small and frequent:** one commit per change that makes sense on its own (a new function, a fixed bug). Not one giant commit at the end of the day.
- **Clear messages, in the imperative:** “Add noise covariance matrix”, “Fix sign in update step”. Avoid “changes”, “stuff” or “fixes”.

- **Check before committing:** `git status` and `git diff` show you exactly what you are about to save.

4. Branches: how they work

The key idea: a branch is not “a part” of the project. When you create a branch from `main`, it contains **everything** that was in `main` at that moment. You add your changes on top.



- **A** and **B** are commits on `main`.
- Two branches start from **B**. Each one has a complete copy of the project up to **B**.
- You work on `feature/simulation` (commits **C**, **D**) and your teammate on `feature/kalman` (commit **F**).
- When the simulation is ready, it is merged into `main` → **E**. Then the Kalman branch is merged → **G**, and `main` has both.

While you work in parallel, **neither of you sees the other's changes** until they come together in `main`. That is exactly what lets you work without stepping on each other.

Branch commands

```
git branch                            # list local branches (* = current one)
git checkout -b feature/simulation    # create a new branch and switch to it
git checkout main                     # go back to main
git push -u origin feature/simulation # upload the branch to GitHub (1st time)
git branch -d feature/simulation      # delete the local branch (once merged)
```

> **Tip.** Before creating a new branch, switch to `main` and run `git pull`. That way you start from the latest version.

How to split the Kalman work into branches

Ideally each branch corresponds to **one specific task** and touches different files from your teammate's. For example:

Branch	What it contains	Files
`feature/geometry`	Detector planes, magnetic field	src/kalmanfit/geometry.py
`feature/simulation`	Generate tracks with noise and multiple scattering	src/kalmanfit/simulation.py
`feature/kalman-filter`	Filter prediction and update	src/kalmanfit/kalman.py
`feature/smoother`	Backward smoothing	src/kalmanfit/kalman.py
`feature/analysis`	Residuals, pulls, χ^2 distribution	src/kalmanfit/analysis.py
`fix/covariance`	Fix one specific bug	whichever it is

- **Short-lived branches:** several branches lasting a few days are better than one huge branch lasting a month. The longer a branch lives, the harder it is to merge.
- **Prefixed names:** `feature/` for new things, `fix/` for fixes, `docs/` for the README or report.
- **Never work directly on main.** Everything goes in through a Pull Request.

5. Merges and Pull Requests

A **Pull Request** is the tidy way to bring your branch into `main`: GitHub shows the changes, your teammate reviews them, and then they are merged with one click.

The full flow, step by step

1	**Start from an up-to-date main**	`git checkout main` `git pull`
2	**Create your branch**	`git checkout -b feature/simulation`
3	**Work and commit**	`git add .` `git commit -m "Add straight-track generator"`
4	**Push the branch**	`git push -u origin feature/simulation`
5	**Open the PR on GitHub**	The “Compare & pull request” button appears. Write what the change does.
6	**Review**	Your teammate looks at the changes and comments if something doesn't add up. You can keep committing and pushing to the same branch: the PR updates itself.
7	**Merge**	Click “Merge pull request”. Your changes are now in main.

8	**Clean up**	`git checkout main`
		`git pull`
		`git branch -d feature/simulation`

Bringing what is already in main into your branch

If your teammate has already merged something you need (for example, the detector geometry for your filter), you bring it into your branch like this:

```
git checkout main
git pull                      # get the latest from GitHub
git checkout feature/kalman-filter
git merge main                # bring those changes into your branch
```

Do this often if your branch lasts several days: that way conflicts stay small and easy to solve.

6. Conflicts

A conflict happens when **both of you have changed the same lines of the same file**. Git doesn't know which one to pick, so it marks it like this:

```
<<<<<<< HEAD
Q = sigma_ms**2 * np.eye(2)
=======
Q = theta0**2 * np.array([[1, 0], [0, 1]])
>>>>>>> main
```

- At the top (between <<<<<<< and =======) is your version; below, the other one.

- Edit the file keeping the correct version (or a mix) and delete the three marker lines.
- Then: `git add file.py` and `git commit`.
- VS Code shows “Accept Current / Incoming / Both” buttons that make this easier.

> **Tip.** The best way to avoid conflicts is to split the work by files or functions, keep branches short, and bring `main` into your branch often.

7. Undoing mistakes

Situation	Command
See the commit history	`git log --oneline --graph`
Discard unsaved changes in a file	`git restore file.py`
Unstage a file (without losing changes)	`git restore --staged file.py`
Change the last commit's message (not pushed yet)	`git commit --amend -m "New message"`
Undo a commit that is already on GitHub	`git revert <commit_id>`
Stop tracking a file you already uploaded (and add it to .gitignore)	`git rm --cached file`
Save half-done changes to switch branch	`git stash` / `git stash pop`

> **Careful.** Avoid `git reset --hard` and `git push --force` unless you really know what you are doing: they can delete work, including your teammate's. To undo something already pushed, always use `git revert`, which creates a new commit that cancels the previous one.

8. Important files in the repo

File	What it's for
`.gitignore`	List of files Git must not upload: Python caches, generated data, temporary figures, LaTeX auxiliary files.
`README.md`	The repo's front page: what the project is, how to install it, how to run it and how it is organised.
`environment.yml`	The conda/Miniforge environment (includes the Python version). Created with `conda env create -f environment.yml`.
`requirements.txt`	The same for people using pip: `pip install -r requirements.txt`.

- When someone adds a new library, they add it to **both** dependency files in the same commit.
- If `environment.yml` changes, the others update their environment with `conda env update -f environment.yml`.
- Don't upload heavy data or results: they are regenerated with the scripts.

Suggested structure

```
kalman-fit/
├── README.md
├── environment.yml
├── requirements.txt
├── .gitignore
├── src/kalmanfit/    geometry.py, simulation.py, kalman.py, analysis.py
├── tests/            checks (e.g. noise-free track -> exact fit)
├── notebooks/        exploration and plots
├── scripts/          scripts that produce the final results
└── report/           LaTeX report and figures
```

9. Command cheat sheet

Command	What it does
`git clone <url>`	Download the repo for the first time
`git status`	See what has changed and which branch you are on
`git diff`	See the changes line by line
`git add <file> / git add .`	Stage changes for the commit
`git commit -m "message"`	Save a snapshot with the staged changes
`git push`	Upload commits to GitHub
`git pull`	Download new commits from GitHub
`git branch`	List branches
`git checkout -b <branch>`	Create a branch and switch to it
`git checkout <branch>`	Switch branch
`git merge <branch>`	Bring the changes from another branch into the current one
`git log --oneline --graph`	See the history in compact form
`git remote -v`	Check which GitHub repo it is connected to
`git stash / git stash pop`	Park half-done changes and get them back

10. Golden rules for working as a pair

- `main` **always works.** Nothing goes into `main` without a Pull Request. You can protect it in Settings → Branches → Add rule.
- `git pull` **before you start working every day.**
- **One branch per task**, with a clear name and a short life.
- **Split up different files or functions** so you don't step on each other.
- **Small commits** with messages that explain what changes.
- **Review each other's PRs:** it's how you both understand all the code (and you'll be asked about it at the defence).
- **Communicate:** before starting a branch, say what you are going to touch.
- **If something breaks, don't force push.** Ask for help or use `git revert`.